

Atividade Pratica - Analise de desempenho criptografico

Texto cifrado no experimento: "RSA: algoritmo dos professores do MIT: Rivest, Shamir e Adleman".
Cada iteracao registrou o tempo total (incluindo geracao da chave e cifragem). Foram feitas 5 iteracoes para cada algoritmo.

Resultados e media de tempo por algoritmo

Algoritmo	It. 1 (ms)	It. 2 (ms)	It. 3 (ms)	It. 4 (ms)	It. 5 (ms)	Media (ms)
RSA-1024	56.310	57.722	79.630	73.025	47.468	62.831
RSA-2048	33.842	51.408	315.619	265.634	369.860	207.273
RSA-4096	6361.508	1589.701	1891.579	2541.969	4996.807	3476.313
RSA-8192	52760.012	10315.365	34523.690	24138.270	20377.506	28422.969
AES-128	1405.530	2.633	4.798	4.565	4.773	284.460
AES-256	0.049	0.014	0.015	0.017	0.013	0.022

Configuracao do sistema utilizado

Item	Valor
Sistema operacional	macOS-26.4.1-arm64-arm-64bit
Processador	Apple M2
Memoria RAM	8.0 GB
Armazenamento	Tamanho: 228Gi Usado: 12Gi Livre: 24Gi

Registro das iteracoes (printscreen)

Iteracao 1

```
=== Iteracao 1 ===
RSA-1024 | 56.310 ms | cifra: 128 bytes
RSA-2048 | 33.842 ms | cifra: 256 bytes
RSA-4096 | 6361.506 ms | cifra: 512 bytes
RSA-8192 | 52760.012 ms | cifra: 1024 bytes
AES-128 | 1405.530 ms | cifra: 64 bytes
AES-256 | 0.049 ms | cifra: 64 bytes

Resultado registrado em: /Users/vinitrevisan/Documents/pucpr/4semestre/seg-informacao/analise-criptografico/resultados.csv
```

Iteracao 2

```
=== Iteracao 2 ===
RSA-1024 | 57.722 ms | cifra: 128 bytes
RSA-2048 | 51.408 ms | cifra: 256 bytes
RSA-4096 | 1589.701 ms | cifra: 512 bytes
RSA-8192 | 10315.365 ms | cifra: 1024 bytes
AES-128 | 2.633 ms | cifra: 64 bytes
AES-256 | 0.014 ms | cifra: 64 bytes

Resultado registrado em: /Users/vinitrevisan/Documents/pucpr/4semestre/seg-informacao/analise-criptografico/resultados.csv
```

Iteracao 3

```
=== Iteracao 3 ===
RSA-1024 | 79.630 ms | cifra: 128 bytes
RSA-2048 | 315.619 ms | cifra: 256 bytes
RSA-4096 | 1891.579 ms | cifra: 512 bytes
RSA-8192 | 34523.690 ms | cifra: 1024 bytes
AES-128 | 4.798 ms | cifra: 64 bytes
AES-256 | 0.015 ms | cifra: 64 bytes

Resultado registrado em: /Users/vinitrevisan/Documents/pucpr/4semestre/seg-informacao/analise-criptografico/resultados.csv
```

Iteracao 4

```
=== Iteracao 4 ===
RSA-1024 | 73.025 ms | cifra: 128 bytes
RSA-2048 | 265.634 ms | cifra: 256 bytes
RSA-4096 | 2541.969 ms | cifra: 512 bytes
RSA-8192 | 24138.270 ms | cifra: 1024 bytes
AES-128 | 4.565 ms | cifra: 64 bytes
AES-256 | 0.017 ms | cifra: 64 bytes

Resultado registrado em: /Users/vinitrevisan/Documents/pucpr/4semestre/seg-informacao/analise-criptografico/resultados.csv
```

Iteracao 5

=== Iteracao 5 ===
RSA-1024 | 47.468 ms | cifra: 128 bytes
RSA-2048 | 369.860 ms | cifra: 256 bytes
RSA-4096 | 4996.807 ms | cifra: 512 bytes
RSA-8192 | 20377.506 ms | cifra: 1024 bytes
AES-128 | 4.773 ms | cifra: 64 bytes
AES-256 | 0.013 ms | cifra: 64 bytes

Resultado registrado em: /Users/vinitrevisan/Documents/pucpr/4o semestre/seg-informacao/analise-criptografica/resultados.csv

Codigo fonte utilizado (Python)

```
import argparse
import csv
import os
from pathlib import Path
from time import perf_counter

from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad

TEXT = "RSA: algoritmo dos professores do MIT: Rivest, Shamir e Adleman"

ALGORITHMS = [
    ("RSA-1024", "RSA", 1024),
    ("RSA-2048", "RSA", 2048),
    ("RSA-4096", "RSA", 4096),
    ("RSA-8192", "RSA", 8192),
    ("AES-128", "AES", 128),
    ("AES-256", "AES", 256),
]

def run_rsa(bits: int) -> tuple[float, int]:
    start = perf_counter()
    key = RSA.generate(bits)
    cipher = PKCS1_OAEP.new(key.publickey())
    ciphertext = cipher.encrypt(TEXT.encode("utf-8"))
    elapsed_ms = (perf_counter() - start) * 1000
    return elapsed_ms, len(ciphertext)

def run_aes(bits: int) -> tuple[float, int]:
    start = perf_counter()
    key = get_random_bytes(bits // 8)
    iv = get_random_bytes(16)
    cipher = AES.new(key, AES.MODE_CBC, iv=iv)
    ciphertext = cipher.encrypt(pad(TEXT.encode("utf-8"), AES.block_size))
    elapsed_ms = (perf_counter() - start) * 1000
    return elapsed_ms, len(ciphertext)

def main() -> None:
    parser = argparse.ArgumentParser(description="Executa uma iteracao do experimento cripto.")
    parser.add_argument("--iteration", type=int, required=True, help="Numero da iteracao (1-5).")
    parser.add_argument("--csv", default="resultados.csv", help="Arquivo CSV de saida.")
    args = parser.parse_args()

    out_path = Path(args.csv)
    out_path.parent.mkdir(parents=True, exist_ok=True)

    file_exists = out_path.exists()
    with out_path.open("a", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        if not file_exists:
            writer.writerow(["iteracao", "algoritmo", "tipo", "bits", "tempo_ms", "tamanho_cifra_bytes"])

    print(f"=== Iteracao {args.iteration} ===")
    for name, kind, bits in ALGORITHMS:
        if kind == "RSA":
            elapsed_ms, cipher_len = run_rsa(bits)
        else:
            elapsed_ms, cipher_len = run_aes(bits)

        writer.writerow([args.iteration, name, kind, bits, round(elapsed_ms, 3), cipher_len])
```

```
        print(f"{name:8} | {elapsed_ms:10.3f} ms | cifra: {cipher_len} bytes")

    print(f"\nResultado registrado em: {os.path.abspath(out_path)}")

if __name__ == "__main__":
    main()
```

Referencias

- Python Software Foundation. Python 3.11 Documentation.
- PyCryptodome Documentation: <https://pycryptodome.readthedocs.io/>
- NIST FIPS 197 - Advanced Encryption Standard (AES).
- PKCS #1: RSA Cryptography Specifications.
- GitHub Copilot (IA) como apoio para estruturacao do experimento e do relatorio.